FULL-STACK TO AI ENGINEER PHASE 00 MASTER NOTES

Phase 00 — Developer Foundations & Setup

Web Architecture, HTTP/REST, Ports, CORS, HTTPS/TLS, Terminal, Permissions, Git, and Secret Management

---

Phase 00 — Developer Foundations & Setup Master Notes

> ***Phase 0: Web Architecture, Network Lifecycle, Environment & System Tooling***
> *Covers HTTP, REST, CORS, HTTPS, Terminal, Paths, Permissions, Package Managers, Git, and Secret Management.*

---

1. Web & Network Lifecycle

The URL Journey

1. **DNS Lookup:** Resolves domain to IP address (`api.site.com` $\rightarrow$ `93.184.216.34`).
2. **TCP 3-Way Handshake:** `SYN` $\rightarrow$ `SYN-ACK` $\rightarrow$ `ACK` on target port.
3. **TLS Handshake:** Secure key exchange for HTTPS encryption.
4. **HTTP Exchange:** Plaintext request travels over encrypted tunnel $\rightarrow$ Server processes $\rightarrow$ Response returns.

Common Ports

- **80:** HTTP (unencrypted)
- **443:** HTTPS (encrypted)
- **3000 / 5173:** Local frontend/dev servers (Next.js / Vite)
- **5432:** PostgreSQL
- **27017:** MongoDB
- `EADDRINUSE` **:** Error when two apps attempt to bind to the same port on the same IP.

---

2. REST Architecture & HTTP Status Codes

REST Rules

- **URLs are NOUNS:** `/orders` , `/orders/42` (never `/getOrder` or `/deleteOrder`).

- **HTTP Methods are VERBS:** `GET` (read), `POST` (create), `PUT` (full replace), `PATCH` (partial update), `DELETE` (remove).
- **Idempotency:** Repeating `GET`, `PUT`, or `DELETE` 100 times leaves the exact same server state. `POST` is non-idempotent.

Status Code Families

- **`200 OK`**, **`201 Created`** (POST), **`204 No Content`** (DELETE).
- **`301 Moved Permanently`**, **`304 Not Modified`** (cache hit).
- **`400 Bad Request`**, **`401 Unauthorized`** (Who are you? Log in first), **`403 Forbidden`** (Logged in, but no permission), **`404 Not Found`**, **`422 Unprocessable Entity`** (data validation failed).
- **`500 Internal Server Error`** (backend code crash), **`502 Bad Gateway`** (reverse proxy reached a dead backend), **`503 Service Unavailable`** (overloaded/restarting), **`504 Gateway Timeout`** (upstream database/API timed out).

---

3. CORS & HTTPS (TLS)

CORS (Cross-Origin Resource Sharing)

- **CORS has NO HTTP status code.** Throws `TypeError: Failed to fetch` in JS.
- Enforced **strictly by the browser**, not the server. Postman, curl, and backend servers ignore CORS completely.
- **Origin = Protocol + Domain + Port.** If any one differs (`localhost:3000` $\rightarrow$ `localhost:5000`), it is Cross-Origin.
- **Preflight (`OPTIONS`):** Test request sent before non-simple requests (`PUT`, `DELETE`, `Authorization`, `application/json`).
- **Fixes:** Backend CORS middleware or frontend proxy/rewrites.

HTTPS (TLS)

Guarantees the triad: 1. **Encryption (Confidentiality):** Eavesdroppers cannot read packets. 2. **Integrity:** Anti-tampering checksums. 3. **Authentication (Identity):** Certificate Authority verifies the real domain.

---

4. Terminal, Paths & Permissions

- **Commands:** `pwd`, `ls -la`, `cd ..`, `mkdir -p`, `touch`, `cat`, `grep -rn "keyword" .`, `which node`.
- **Paths:** Absolute (starts from root `/` or `C:\`) vs Relative (starts from `pwd`). Always use `path.join()` in Node.js for cross-platform compatibility.

- **`chmod` Math:** `r = 4`, `w = 2`, `x = 1`.
- `chmod 755`: Owner `rwx` (7), Group `r-x` (5), Others `r-x` (5).
- `chmod 644`: Owner `rw-` (6), Group `r--` (4), Others `r--` (4).
- **Danger:** Never use `sudo npm install -g` (creates root-owned files and grants root access to third-party install scripts).

---

5. Package Managers & Python venv

- **`npm` vs `npx`:** `npm` installs packages to disk; `npx` runs binaries from a temporary cache without permanent installation.
- **`package.json` vs `package-lock.json`:** `package.json` specifies version ranges (`^4.19.0`); `package-lock.json` freezes exact versions and cryptographic hashes. **Both must be committed to Git.** `node_modules/` must NEVER be committed.
- **Python `.venv`:** Node installs packages locally by default (`node_modules`). Python installs globally by default. `.venv` forces Python to create a private local sandbox for the project.

---

6. Git & Secret Management

- **The 3 Stages:** Working Directory $\rightarrow$ `git add` (Staging Area) $\rightarrow$ `git commit` (Local Repository).
- **`git merge` vs `git rebase`:** Merge preserves true non-linear history with a merge commit; rebase replays commits to make a flat linear history. **Rule: Never rebase a shared branch.**
- **`git stash`:** Shelves dirty work so you can switch branches, restored with `git stash pop`.
- **Secrets & `.env`:** `.env` is ignored by Git; `.env.example` is committed as a template.
- **`NEXT_PUBLIC_` Risk:** Next.js bakes `NEXT_PUBLIC_` variables directly into public client JavaScript bundles during build. Never put API secrets or database passwords in `NEXT_PUBLIC_`.
- **Leaked Key Protocol:**
- Revoke / delete key immediately at provider console.
- Rotate keys and update `.env`.
- Inspect billing and audit logs.
- Purge Git history using `git-filter-repo` (deleting in a new commit leaves it in history!).